AIPL: Actor based Intelligent Parallel Language

— 設計から実現 —

第 2 版：ABCL/1 と小林の型システムから、何を採り何を落としたか

児玉靖司

法政大学経営学部

yass@hosei.ac.jp

2026 年 8 月 22 日

概要

本稿は、アクターを単位とする型付き並行言語 AIPL（Actor based Intelligent Parallel Language、処理系名 ABCLc+）の設計と実装を、**何を先行研究から採り、何を落とし、なぜそうしたか**を軸にまとめ直したものである。

源流は二つある。一つは ABCL/1（Yonezawa, Briot, Shibayama, OOPSLA 1986）で、三種のメッセージ送信（past / now / future 型）とオブジェクト内部の逐次性を受け継いだ。一方で express mode による割り込み、返信先の一級性、`script` の `where` ガードは落とした。もう一つは小林直樹の型システム研究の系譜で、効果（資源使用解析の素朴版）と「ちょうど一度」の線形性の考え方を採り、交差型による高階モデル検査・多相メソッド・完全な所有権型は落とした。デッドロック自由な型システムは採らず、**期限（deadline）で代用**している。

採否の理由は、いずれも**動く最小プログラムで示す**。落としたものについては「落とすと何が書けなくなるか」を、採ったものについては「採ると何が止められるようになるか」を、実際に処理系へ通した出力とともに述べる。そのうえで最新仕様の代表的なサンプル 10 本を、コード・出力・メリット・デメリットとともに解説する。サンプルはすべて**三つの独立な処理系**（OCaml 版・Python 版・JavaScript 版）で出力が一致することを確認した。

本稿の執筆過程そのものが結果を生んだ。10 本を三実装に通す作業で、アクター生成・文法・出力の取りこぼしに関する実装のバグが 5 件見つかり、うち 3 件は**正であるはずの OCaml 版**のものであった。また `become` については、ABCL/1 に無い機構でありながら現状の型付けを単独で無効にしていることが分かり、Akka Typed と同じ制限 —- 置換先は置換前が受け取れたメッセージをすべて受け取れること —- を型検査で課すことにした。

目次

1 はじめに 4

2 背景 4

2.1 ABCL/1 4

2.2 小林らの型システム研究 4

2.3 他の並行オブジェクト言語 4

3 ABCL/1 から採ったもの 5

3.1 三種のメッセージ送信 5

3.2 オブジェクト内部の逐次性 . . . 5
3.3 返信という考え方 . . . 5

4 ABCL/1 から落としたもの 5
4.1 express mode による割り込み . . . 6
4.2 返信先の一級性 . . . 6
4.3 `script` の `where` ガード . . . 6

5 小林研究から採ったもの 7
5.1 効果 —- 資源使用解析の素朴版 . . . 7
5.2 期限 —- サイズ型の粗い代用 . . . 8
5.3 線形性の考え方 —- `reply` は高々一度 . . . 8

6 小林研究から落としたもの 9
6.1 交差型による高階モデル検査 . . . 9
6.2 多相メソッド . . . 9
6.3 完全な所有権の型システム . . . 9
6.4 デッドロック自由な型システム（採らず、期限で代用） . . . 9

7 採否の対応表 10

8 `become` —- どちらの源流にも無いもの 10
8.1 無制限だと型が嘘になる . . . 10
8.2 Akka Typed と同じ制限を課す . . . 11

9 代表的なサンプルプログラム 11
9.1 s01 —- アクターの基本 . . . 11
9.2 s02 —- 三種の送信 . . . 12
9.3 s03 —- `reply` の規律 . . . 12
9.4 s04 —- 効果 . . . 12
9.5 s05 —- 期限 . . . 13
9.6 s06 —- 実時間側の防壁 . . . 13
9.7 s07 —- `select` . . . 13
9.8 s08 —- `become` . . . 14
9.9 s09 —- アクターを引数に渡す . . . 14
9.10 s10 —- 三段のパイプライン . . . 14
9.11 10 本を通して見えること . . . 15

10 三つの処理系と、揃っていることの確かめ方 15
10.1 本稿の 10 本を三実装に通して見つかったもの . . . 16
10.2 現在の状態 . . . 16

11 考察 16
11.1 採否の判断は一貫していたか . . . 16

11.2 「正」の実装という考え方の限界 17
11.3 型・効果・期限という三点セットの評価 17
11.4 残っている課題 17

12 **おわりに** 17

1　はじめに

AIPL は二つの目的のために作られている。**分散 AI エージェント**の記述と、**実時間 OS** の記述である。前者は「機外のモデルを呼ぶので、いつ返るか分からない」世界であり、後者は「期限を守れないことが故障である」世界である。両方をアクターとメッセージで書きたい。そして**両者を混ぜてはいけない**。実時間で動くアクターが機外のモデルを同期で待ってはならない。これを規約ではなく型検査で禁じたい —- これが設計を貫く動機である。

言語を一から作るのではなく、二つの先行研究から出発した。ABCL/1 は三種のメッセージ送信を中核に据えた設計であり、この三種はそのまま「待たない／待つ／後で受け取る」という区別に対応する。ただし ABCL/1 は動的型であり、型システムを持たない。その基礎づけを与える試みが、小林直樹らの一連の研究である。

本稿の構成は次のとおりである。§2 で背景を述べ、§3 と §4 で ABCL/1 から採ったもの・落としたものを、§5 と §6 で小林研究から採ったもの・落としたものを、それぞれ動くプログラムとともに述べる。§7 に採否の対応表を置く。§8 は、どちらの源流にも無い `become` の扱いを論じる。§9 が代表的なサンプル 10 本、§10 が三つの処理系、§11 が全体の考察である。

本稿のプログラムはすべて実際に処理系へ通した。出力は貼り付けたものではなく、実行して得たものである。

2　背景

2.1　ABCL/1

ABCL/1 は並行オブジェクトを計算の単位とする言語である。オブジェクトは自分のメールボックスを持ち、到着したメッセージを一つずつ処理する。この「一度に一つ」がオブジェクト内部の逐次性を保証し、それゆえフィールドへの排他制御を書かなくてよい。

中核は四つ —- 三種のメッセージ送信、`script` によるパターン受信（`where` ガードつき）、express mode による割り込み、返信先（reply destination）の一級性である。

2.2　小林らの型システム研究

小林と米澤の一連の研究は、線形論理を土台として並行オブジェクトの型理論的基礎を与えた。重要なのは、AIPL が直面する問題の多くがすでにこの系譜で扱われていることである—- 線形型（「ちょうど一度」）、デッドロック自由な型システム、π 計算の汎用型システム、資源使用解析、高階モデル検査と交差型の等価性、述語抽象と CEGAR、精製型・サイズ型による計算量。

2.3　他の並行オブジェクト言語

比較の座標として次を置く。ABCL/f は同じ ABCL 系で型を持つ先行例で、**future だけを残して型を付ける**という判断をとった。Erlang/OTP は `gen_statem` により状態遷移を書くが、コールバックが受け取るメッセージ集合は固定である。Akka は classic では `context.become` が無型であったが、**Akka Typed で `Behavior[T]` の `T` を固定する形に作り直した**。この最後の一点は §8 で効いてくる。

3　ABCL/1 から採ったもの

3.1　三種のメッセージ送信

最も本質的な継承である。

AIPL の書き方	ABCL/1	意味	式か文か
`send t.m(a);`	past	送って待たない	文
`now t.m(a) timeout` n `else` e	now	送って返信を待つ	式
`future t.m(a)` / `await f ...`	future	引換券を受け取る	式

```
class Calc {
  method twice(x) : int { reply(x * 2); }
  method slow(x)  : int { reply(x + 100); }
}
var c = new Calc();
send c.twice(1);                                          // 待たない
print("now    = " ++ (now c.twice(21) timeout 100 else 0)); // 待つ
var f = future c.slow(5);                                 // 引換券
print("future = " ++ (await f timeout 100 else 0));       // あとで受け取る
```

```
now    = 42
future = 105
```

■なぜ採ったか。 この三分割は、そのまま AIPL が必要とする区別である。実時間側は「待たない」を多用し、エージェント側は「後で受け取る」を使う。ABCL/f は future だけを残したが、AIPL は三種すべてを残した。そのぶん型付けは面倒になる —- `send` は返信を受け取らないので戻り値型を要求せず、`now` は式なのでその場に戻り値型が要り、`future` は τ を返すメソッドに `future`$\langle\tau\rangle$ を与える。この面倒は、三種の区別が設計上必要だという判断に対する代価である。

3.2　オブジェクト内部の逐次性

アクターはメッセージを一つずつ処理する。フィールドへの代入に排他制御が要らないのはこのためで、§5 で述べる効果 `mut` が意味を持つのもこの前提のうえである。

3.3　返信という考え方

値は `return` ではなく `reply` で返す。これは ABCL/1 の返信をそのまま受けたものだが、静的型を載せるうえでは厄介な選択である。`return` なら関数の型が戻り値型を決めるが、`reply` は本体のどこにでも書ける文だからである。AIPL はメソッドごとに戻り値型を表す型変数 ρ を置き、本体の各 `reply(e)` と単一化することで推論する。

4　ABCL/1 から落としたもの

三つとも「入れられない」のではなく、**入れると証明の形が変わる**という理由で落とした。

4.1　express mode による割り込み

ABCL/1 は ordinary モードとは別の待ち行列を持ち、express メッセージは実行中の処理を中断して割り込む。

■落とした理由。 §3 の逐次性が破れる。「メソッド本体の実行が最後まで途切れない」という前提のうえに、フィールドの不変条件も、効果の集計も、型健全性の証明も立っている。割り込みを入れると、本体の途中で状態が観測されうるので、これらをすべて「割り込み点で成り立つ不変条件」に書き直す必要がある。

■落とすと何が書けなくなるか。 中止・優先処理・監視である。AIPL では `select` と期限で近いことを書くが、**実行中の処理を止めることはできない**。できるのは「次に何を受けるか」を選ぶことだけである。

4.2　返信先の一級性

ABCL/1 では *now* 型送信で暗黙に作られる返信先が値であり、他へ渡せる。つまり「A が受けた依頼に B が代わりに答える」委譲が書ける。

■落とした理由。「ちょうど一度だけ返信する」を保てないからである。AIPL は `reply` の回数を**構文の走査**で数えている —- 本体に現れる `reply` の上界を数え、2 以上なら拒否する。これは返信先が暗黙である限り成立するが、値として渡せるようにした途端、渡された先で何回返されるかは走査では分からない。線形型を入れれば解けるが、それは §6 の話になる。

■副産物。 この判断には思わぬ効果があった。`self` と `sender` は**送信の宛先としてしか書けず、値ではない**。

```
send self.h(1);          -> OK
send e.take(self, x);    -> PARSE_ERROR
```

そのため、二者が互いに `now` で待ち合う循環を、現在の構文では素直に組み立てられない。デッドロック自由性を型で保証しているわけではないが、**一級性を落としたことが、待ち合いを作りにくくもしている**。

4.3　`script` の `where` ガード

ABCL/1 は受信のパターンに `where` 節を書け、「在庫があるときだけ購入メッセージを受理する」といった記述ができた。AIPL の `select` はパターンをメソッド名と引数の並びに限り、`where` を持たない。

■落とした理由。 進行定理に枝が一本増えるからである。どのガードも成立せず受理できない構成が現れうるので、「終状態」「簡約できる」に加えて「受理できるメッセージが無い」を扱う必要が出る。さらにガードの純粋性という新しい要求も生じる（ガードが状態を変えてはならない）。

■落とすと書き方がどう変わるか。 受理してから本体で分岐する。

```
class Shop {
  var stock = 2;
  method buy(n) : string !{mut} {
    if (stock >= n) { stock = stock - n; reply("ok, left=" ++ stock); }
```

```
    else { reply("sold out"); }
  }
}
var s = new Shop();
print(now s.buy(1) timeout 200 else "?");
print(now s.buy(5) timeout 200 else "?");
```

```
ok, left=1
sold out
```

違いは**受理してしまう**点にある。where なら「受理しない＝送り主は待ち続ける（あるいは他が受理する）」であったのが、AIPL では「受理して、断りの返信をする」になる。在庫が戻るまで待たせたい場合、その待ちはプログラマが書くことになる。

5　小林研究から採ったもの

5.1　効果 —- 資源使用解析の素朴版

メソッドは自分が世界に及ぼす作用を宣言できる。効果は 8 種類の固定語彙である。

mut	自分の状態を変える	net	ネットワークに出る
time	時間に依存する	ai	言語モデルを呼ぶ
io	入出力	fs	ファイルを触る
mem	動的に割り付ける	log	ログを出す

```
class Store {
  var n = 0;
  method bump()  : int !{mut}  { n = n + 1; reply(n); }   // 状態を変える
  method peek()  : int !{}     { reply(n); }              // 純粋
  method show()  : unit !{log} { print("n=" ++ n); }
}
```

書かなければ本体から推論し、書いた場合は本体から推論した集合と照合する。足りなければ型エラーになる。

■採った理由。　これが §1 の動機に対する直接の答だからである。実時間側のアクターに !{time, log} と書いておけば、そこから ai や net を持つメソッドを**同期で**呼ぶことが型エラーになる。

```
[Type error] method Ctrl.viaNow declares {log, time}
             but its body has {ai, log, net} (missing {ai, net})
```

小林の資源使用解析に比べれば素朴である —- 効果は**順序を持たない集合**で、「開いたら閉じる」のような性質は書けない。そこは今後の課題として残っている（§11）。

■規律の細部。　効果を引き継ぐのは**待つ**呼び出しだけである。send は送って待たないので引き継がない。この判断は明示的なもので、逆に send も引き継ぐようにして手元の .aipl 543 本を通してみたところ、落ちたのは**この規律を説明している例題ただ一本**であった。

5.2 期限 —- サイズ型の粗い代用

now と await には期限を書ける。

```
class Slow {
  method fast(x) : int { reply(x); }
  method never(x) : int { while 1 > 0 do { } reply(x); }   // 返ってこない
}
var s = new Slow();
print("fast  = " ++ (now s.fast(42) timeout 200 else -1));
print("never = " ++ (now s.never(7) timeout 200 else -1));
```

```
fast  = 42
never = -1
```

■採った理由と、その位置づけ。 小林のサイズ型・精製型は応答時間の上界を**型で**述べる。AIPL の期限はそこまで行かず、実行時に打ち切るだけである。しかし効果は非常に大きい —- 期限のない待ちを構文から外すと、待ちが「残り時間を減らす簡約」として進むので、進行定理から「未解決 future の待ちでブロックする」枝が消える。系としてデッドロック自由が言える。

代償は明確である。まず停止性は言えない。そして**デッドロックが消えるのではなく、有限時間の失敗に変わる**。

```
class Never  { method wait_forever(x) : int !{} { while 1 > 0 do { } reply(x); } }
class Caller {
  var n = new Never();
  method go(x) : int !{} { reply(now n.wait_forever(x) timeout 200 else -1); }
}
var c = new Caller();
print("go = " ++ (now c.go(5) timeout 900 else -999));
```

```
go = -1
```

詰まらない。しかし正しくもない。−1 が返るだけである。

5.3 線形性の考え方 —- reply は高々一度

線形型そのものは採っていないが、「ちょうど一度だけ使う」という規律は reply に持ち込んでいる。

- **線形性**：一つのメッセージに対する reply は高々一度。
- **被覆**：戻り値型を宣言したら、すべての実行経路で reply に到達する。

```
[Type error] method f may reply more than once on some path;
             a method must reply at most once
[Type error] method f is declared to reply with int,
             but some execution path does not reply
```

ただし実現は**構文の走査**であって型ではない。そのため §4 で述べたとおり、返信先を値にした瞬間に無力になる。

6　小林研究から落としたもの

6.1　交差型による高階モデル検査

交差型による型付けと μ 計算のモデル検査の等価性は、検証器を自作せずに借りるための強力な道具である。

■落とした理由。 型推論が一般に決定不能であり、AIPL の対象（アクターと通信）に対して必要な表現力を超えている。加えて、反例が AIPL の言葉で出てこない —- 検査器が返すのは高階再帰スキームの反例であって、「どのアクターがどのメッセージで詰まるか」ではない。利用者に見せられない反例は、検証としては使えても道具としては使えない。

6.2　多相メソッド

メソッドは単相である。同じメソッドを二つの型で使うことはできない。

```
class Id { method id(x) : any { reply(x); } }
var i = new Id();
print(now i.id(1) timeout 100 else 0);
print(now i.id("s") timeout 100 else "?");
```

```
[Type error] line 3, col 7: type mismatch
```

■落とした理由。 戻り値型を表す ρ を型スキームに含めると、**呼び出しごとに ρ が新しくなる**。すると「このメソッドはどの型を返すか」という制約が呼び出し側へ伝わらず、`reply` の地点と結ばれなくなる。ρ はクラスのメソッド表に属する量であって、多相化の対象ではない。単相のままにしておくのが、`reply` からの推論と型健全性の証明の双方に対して整合的である。

■代価。 上の `Id` のような汎用のアクターが書けない。実際には `any` を使って型検査を素通りさせることになるが、それは検査を放棄しているだけである。

6.3　完全な所有権の型システム

Rust 相当の借用検査は作らない。CHC への還元に必要な範囲（別名が無いこと）は、アクターがメッセージでしか相互作用しないという性質から既に成立しているためである。

6.4　デッドロック自由な型システム（採らず、期限で代用）

通信の依存関係に順序を入れて循環待ちを型で排除する手法は、AIPL の課題にそのまま噛み合う。しかし現状は採っていない。

■採らなかった理由。 義務レベルの推論が利用者に説明しにくいこと、そして期限による代用で**工学的には**詰まらなくなるためである。ただし §5 で示したとおり、代用は保証ではない。将来は「明示宣言のみ」の形から始めるのが現実的だと考えている。

7　採否の対応表

機構	採否	理由	AIPL での姿
ABCL/1 から			
三種の送信	採る	「待たない／待つ／後で」は設計上必要な区別	`send` / `now` / `future`+`await`
内部の逐次性	採る	排他制御を書かずに済む。証明の土台	一度に一つ処理
`reply` による返信	採る	源流の考え方。型付けは ρ で解決	`reply(e)`、戻り値型は推論または注釈
express mode	落とす	逐次性が破れ、証明を全面的に書き直す必要	無し。`select` ＋期限で近いことのみ
返信先の一級性	落とす	「ちょうど一度」を構文走査では守れない	無し。`self`/`sender` は値でない
`where` ガード	落とす	進行定理に枝が増え、ガードの純粋性が要る	`select` は名前と引数のみ。本体で分岐
小林らの研究から			
資源使用解析	採る（素朴版）	実時間側とエージェント側を分ける唯一の手段	8 種の効果集合。順序は持たない
サイズ型・精製型	採る（粗い代用）	上界を型で言う代わりに実行時に打ち切る	`timeout` n `else` e
線形型の考え方	採る（構文で）	「ちょうど一度」は返信の要	`reply` の線形性・被覆検査
線形型（型として）	落とす	返信先を一級にしないので当面不要	無し。一級化するなら必須
デッドロック自由な型	落とす	義務レベルの推論が説明しにくい	期限で代用。保証ではない
交差型モデル検査	落とす	推論が決定不能。反例が言語の言葉で出ない	無し
多相メソッド	落とす	ρ をスキームに含めると reply と結べない	単相のみ
完全な所有権型	落とす	別名の無さは既に成立	無し
どちらの源流にも無いもの			
`become`	採る（型で縛る）	Hewitt 系 actor 由来。無制限だと型が嘘になる	置換先は置換前の全メッセージを受けること

8　become ―- どちらの源流にも無いもの

`become` は ABCL/1 には無い。ABCL/1 は状態変数と `script` の制約で同じことを表現していた。AIPL の `become` は Hewitt 系アクターモデル由来である。

8.1　無制限だと型が嘘になる

```
class Closed2 {
  method state() : unit !{log} { print("closed"); }
  method open()  : unit !{mut, mem, log} { become Opened2(); }
}
class Opened2 {
  method opened() : unit !{log} { print("opened"); }   // state も open も無い
}
var d = new Closed2();
send d.open();
send d.state();        // become 後には存在しないメソッド
```

修正前、これは**型検査を通った**。d の静的な型は actor(Closed2) であり、そこに state はある。しかし実行時には無い。つまり become は、アクター型の区別という規律を単独で無効にしていた。

8.2 Akka Typed と同じ制限を課す

他言語の扱いを見ると、**型のある設定では一貫して制限が課されている**。Akka は classic の無型 context.become を、Akka Typed で Behavior[T] の T を固定する形に作り直した。Erlang/OTP の gen_statem も、状態は変わってもメッセージ集合は固定である。

そこで AIPL でも、**置換先は置換前が受け取れたメッセージをすべて受け取れること**を型検査で要求することにした。

```
[Type error] become Opened2: Opened2 does not accept state, open,
             which Closed2 accepts
```

これを満たす書き方は、状態機械のすべての状態ですべてのメッセージを受けることである。

```
class Closed {
  method state() : unit !{log} { print("closed"); }
  method open()  : unit !{mut, mem, log} { print("opening"); become Opened(); }
}
class Opened {
  method state() : unit !{log} { print("opened"); }
  method open()  : unit !{log} { print("already open"); }   // 受けるが何もしない
}
```

所見 1. 表現力の観点では become は不要である —- 状態フィールドと本体の分岐で同じことが書ける。残す価値は**状態機械としての読みやすさ**にあり、その価値を保ったまま型を守る方法が、上の制限である。手元の 543 本にこの制限を課した結果、**影響を受けたファイルは 0 本**であった。

9 代表的なサンプルプログラム

最新仕様の代表として 10 本を挙げる。出力はすべて実行して得たものであり、**三つの処理系すべてで一致する**ことを確認している（§10）。

9.1 s01 —- アクターの基本

```
class Greeter {
  var count = 0;
  method init(name) { print("hello, " ++ name); }   // new のとき自動で呼ばれる
  method tick() : unit !{mut, log} {
    count = count + 1;
    print("tick " ++ count);
  }
}
var g = new Greeter("AIPL");
send g.tick();
send g.tick();
```

```
hello, AIPL
tick 1
tick 2
```

示すもの：クラス・フィールド・init・非同期送信・文字列連結。++ が連結で + は数値専用である。**メリット**：状態を持つ並行部品が 10 行で書け、排他制御が要らない。**デメリット**：send は結果を返さないので、順序に依存する処理を書くと**送った順にしか保証がない**。

9.2 s02 —- 三種の送信

コードと出力は §3 に示した。**メリット**：待つ・待たないを**呼ぶ側**が選べる。同じメソッドを実時間側は send で、エージェント側は future で使える。**デメリット**：三種あるぶん、初学者はどれを使うか迷う。now を安易に選ぶと待ちが連鎖する。

9.3 s03 —- reply の規律

```
class Router {
  method route(k) : string {
    if (k > 0) { reply("positive"); } else { reply("non-positive"); }
  }
}
var r = new Router();
print(now r.route(5)  timeout 100 else "?");
print(now r.route(-1) timeout 100 else "?");
```

```
positive
non-positive
```

示すもの：戻り値型を宣言すると、全経路での reply が義務になる。else を消せば型エラーになる。**メリット**：「返信を忘れて相手が待ち続ける」が静的に止まる。**デメリット**：判定が構文の走査なので保守的である。while の本体にしか reply が無い場合、0 回実行されうるとみなして拒否する。

9.4 s04 —- 効果

コードは §5 に示した。

```
1
1
n=1
```

メリット：!{} と書けたメソッドは純粋だと分かるので、自由に複製・再配置できる。分散配置の判断が**型から機械的に決まる**。**デメリット**：語彙が 8 個に固定で、利用者が増やせない。また順序を持たないので「取得したら解放する」が書けない。

9.5 s05 —- 期限

コードと出力は §5 に示した。**メリット**：どんな相手でも待ちが有限で終わる。実時間側の設計が「最悪でも n ミリ秒」で組める。**デメリット**：else の値が**正常値と区別できない**。上の例の -1 は「失敗」を意味するが、型は int のままである。失敗を型で表すには直和型が要る。

9.6 s06 —- 実時間側の防壁

```
class Planner {                          // エージェント側
  method plan(g) : string !{ai, net} { reply(ai_call("plan " ++ g)); }
}
class Servo {                            // 実時間側。ai も net も持たない
  var p = new Planner();
  method step() : unit !{time, log} {
    send p.plan("grasp");                // 送るだけなら許される
    print("servo step");
  }
}
var s = new Servo();
send s.step();
```

```
servo step
```

示すもの：本稿の看板。send を now p.plan(...) に変えると、Servo.step が ai, net を宣言していないので型エラーになる。**メリット**：設計上の誤りが規約ではなく型検査で止まる。**デメリット**：この保証は**注釈を書いた場合にだけ**効く。注釈を省くと本体から推論されるので、誤りは誤りのまま通る。防壁は「宣言する」という利用者の行為に依存している。

9.7 s07 —- select

```
class Worker {
  method job(k) : string { reply("direct:" ++ k); }
  method serve() : unit !{log} {
    print("waiting");
    select {
      case job(k) -> { print("got " ++ k); reply("via select:" ++ k); }
      timeout 500 -> { print("timed out"); }
    }
  }
```

```
}
var w = new Worker();
send w.serve();
print(now w.job(1) timeout 800 else "?");
```

```
waiting
got 1
via select:1
```

示すもの：case 本体の reply は、囲む serve ではなく**受理した job への返信**になる。**メリット**：待ち受けと処理が一箇所に書ける。**デメリット**：where が無いので受理条件を状態で絞れない。また reply の帰属が直感に反しやすく、被覆検査もここで別扱いになる。

9.8 s08 —- become

コードと出力は §8 に示した。

```
closed
opening
opened
```

メリット：状態遷移が**クラスとして**読める。if の連なりより状態機械の構造が見える。**デメリット**：すべての状態がすべてのメッセージを受ける義務が生じるため、状態が増えると受けるだけのメソッドが増える。

9.9 s09 —- アクターを引数に渡す

```
class Device { method ping() : unit !{log} { print("pong"); } }
class Driver {
  method attach(d: Device) : unit !{log} {   // Device 以外を渡すと型エラー
    print("attached");
    send d.ping();
  }
}
var dev = new Device();
var drv = new Driver();
send drv.attach(dev);
```

```
attached
pong
```

示すもの：引数に型注釈を書くとクラスが決まり、Device 以外を渡すと actor type mismatch になる。**メリット**：ドライバのような「相手を受け取って使う」部品が型安全に書ける。**デメリット**：注釈は**必須**である。省くと引数の型が決まらず、送信そのものが型エラーになる。また注釈は**クラス名**であってインタフェースではないので、「ping を持つ何か」を受け取ることはできない。

9.10 s10 —- 三段のパイプライン

```
class Source { method get(i) : int !{} { reply(i * 10); } }
```

```
class Scale  {
  var src = new Source();
  method at(i) : int !{} { reply((now src.get(i) timeout 100 else 0) + 1); }
}
class Sink {
  var sc = new Scale();
  var total = 0;
  method run(n) : int !{mut, log} {
    var i = 0;
    while i < n do {
      total = total + (now sc.at(i) timeout 100 else 0);
      i = i + 1;
    }
    print("total=" ++ total);
    reply(total);
  }
}
var s = new Sink();
print("sum = " ++ (now s.run(4) timeout 1000 else -1));
```

```
total=64
sum = 64
```

示すもの：フィールドに new でアクターを持ち、now を二段重ねる実用的な形。**メリット**：段ごとに独立したアクターなので、後から段を差し替えたり分散配置したりできる。**デメリット**：now の連鎖は**待ちの連鎖**である。各段の期限が積み上がるので、外側の期限は内側の合計より大きくしなければならない。上の例で外側を `timeout 100` にすると -1 になる。

9.11　10 本を通して見えること

	効いている仕組み	代価
s01–s03	型（reply の規律）	判定が保守的
s04, s06	効果	語彙が固定・注釈依存
s05, s10	期限	失敗が正常値と区別できない
s07–s09	アクターの構造	受理条件・注釈・インタフェースの制限

三つの仕組みはそれぞれ独立に理解できるが、**噛み合って初めて言えること**が s06 である。効果だけでも期限だけでも、あの誤りは止まらない。

10　三つの処理系と、揃っていることの確かめ方

AIPL には実装が三つある ── 正となる OCaml 版（手書き 7,289 行）、Python の CLI 実装（14,443 行）、ブラウザで動く JavaScript 実装（5,709 行）である。三つ作る理由は二つある。JS 版はブラウザで動くので処理系を配らずに試してもらえること、そして**三つ揃えること自体が仕様の検査になる**ことである。

確かめ方は二本立てである。`run_all_impls.sh` が**通るプログラムの出力が一致するか**を見て、`run_reject_tests.sh` が**通ってはいけないプログラムが止まるか**を見る。後者は 16 本の「弾かれるべ

きプログラム」からなる。

10.1　本稿の 10 本を三実装に通して見つかったもの

執筆のために 10 本を三実装へ通したところ、実装のバグが 5 件出た。

1. **フィールドを `new` で初期化するとアクターが動かない（OCaml 版）**。アクター生成のコードが二箇所に重複して書かれており、フィールドの初期化が `New` を扱えない評価器を呼んでいた。生成が例外で止まり、そのアクターはメッセージを一つも処理しないまま消えていた。s06 と s10 が丸ごと無出力だったのはこれである。
2. **同じ症状（JS 版）**。フィールドの初期化に `New` の枝が無く、フィールドがアクターではなく数値 `0` になっていた。
3. **文法が三つ足りない（JS 版）**。`while ... do`、`become`、単項マイナスが文法に無かった。いずれも現行の文法要約に載っている構文である。
4. **スクリプト終了時に出力を取りこぼす（OCaml 版）**。20 回中 7 回、行が欠けていた。メールボックスが空になり、処理中のメッセージも無くなるまで待ってから抜けるようにした。
5. **`send` の効果の扱いが三者三様**。§5 で述べた規律に対し、Py 版と JS 版が引き継ぐ側になっていた。

所見 2. 5 件のうち 3 件が**正であるはずの OCaml 版**のものであった。「正」は仕様の基準という意味であって、実装が正しいという意味ではない。そして 1 と 2 は**同じ形のバグ**である —-「アクター生成の本筋」と「フィールド初期化」が別の道を通っており、後者だけが `new` を扱えなかった。独立に書かれた三つの実装が同じ場所で同じ間違いをしたことは、これが**実装者の不注意ではなく設計の穴**であることを示している。フィールド初期化を式の評価と別扱いにしたこと自体が誤りだった。

10.2　現在の状態

10 本すべてが三実装で出力一致する。ガイドの 8 本も一致する。弾かれるべき 16 本については、48 枠のうち 11 枠が期待と食い違い、その内訳は JS 版 8・Py 版 3 である（OCaml 版は 16 本すべてを弾く）。残りはいずれも「OCaml 版にはあるが他の実装に無い検査」であり、言語仕様の穴ではなく追随の遅れである。

11　考察

11.1　採否の判断は一貫していたか

振り返ると、落としたものには共通の理由がある —- **証明の形が変わるから**である。express mode は逐次性を、`where` は進行定理の枝を、返信先の一級性は「ちょうど一度」を、それぞれ壊す。交差型モデル検査と多相メソッドも、決定可能性と ρ の扱いという形で証明の側の理由で落ちている。

一方、採ったものには別の共通点がある —- **目的から要求されたから**である。三種の送信も、効果も、期限も、「分散 AI エージェントと実時間 OS を同じ言語で書き、混ぜない」という要求から出ている。

この二つの基準は、たまたま噛み合っている場面と、そうでない場面がある。噛み合っているのが効果と期限で、目的にも証明にも効いた。噛み合っていないのが返信先の一級性で、**目的からは欲しいのに証**

明の側から落とした。§6 で述べたとおり、これは線形型を入れれば取り戻せる。言い換えれば、AIPL の次の一歩は**証明の側の制約を緩める**方向にある。

11.2 「正」の実装という考え方の限界

§10 の所見は、本稿で最も実務的な発見である。三つの実装のうち一つを「正」と決め、他を追随させる運用をとってきた。これは仕様が一つに定まる利点がある一方で、**正の実装が間違えたとき、それを検出する仕掛けが無い**。

今回、効果伝播の穴が OCaml 版だけのものであったこと、フィールド初期化のバグが OCaml 版と JS 版に独立に存在したことは、どちらも「三つ並べて、通らないはずのものを通してみる」ことで初めて出た。出力の比較では出ない。正しいプログラムでは、どの実装も正しく動くからである。

ここから言えるのは、多実装を持つことの価値は**移植性ではなく、仕様の検査にある**ということである。そしてそのためには、正しいプログラムの集合と同じだけ、**誤ったプログラムの集合を資産として持つ**必要がある。

11.3 型・効果・期限という三点セットの評価

10 本を通して見ると、三つの仕組みの効き方には差がある。

型は最もよく効いている。`reply` の規律とアクター型の区別は、書き間違いの多くをその場で止める。代価は判定の保守性で、正しいのに拒否される場合がある。

効果は、効くときは非常に効くが**注釈依存**である。書かなければ推論されるだけで、誤りは通る。これは「効果を書かないと損をする」仕組みが無いためで、たとえば公開するアクターには注釈を義務づけるといった運用が要る。

期限は最も安上がりに強い性質を与える。構文を一つ足すだけで進行定理の枝が消える。しかし §5 で見たとおり、これは失敗を消すのではなく**失敗の形を変える**だけである。−1 が返ることと、正しく計算できたことを、型は区別しない。ここが現状で最も弱い箇所だと考えている。

11.4 残っている課題

優先順に三つ挙げる。

1. **効果に順序を入れる**。現在の効果は集合であり、「取得したら解放する」が書けない。小林の資源使用解析の方向で、OS を記述する目的に直結する。
2. **失敗を型で表す**。期限切れの `else` の値が正常値と同じ型であるかぎり、「この値は本当に計算された値か」をプログラムが問えない。
3. **線形型による返信先の一級化**。ABCL/1 から落とした三つのうち唯一「型の問題」であるものが解け、委譲が書けるようになる。

12 おわりに

本稿は AIPL の設計と実装を、採否の理由という軸でまとめ直した。ABCL/1 からは三種の送信・逐次性・返信を採り、express mode・返信先の一級性・`where` を落とした。小林の系譜からは効果と期限と線形性の考え方を採り、交差型モデル検査・多相メソッド・完全な所有権型を落とし、デッドロック自由な型

は期限で代用した。どちらの源流にも無い `become` は、Akka Typed と同じ制限を課したうえで残した。

最後に、執筆そのものから得たことを一つ記しておく。サンプルを 10 本書いて三つの実装に通す ——ただそれだけの作業で、実装のバグが 5 件出た。うち 3 件は正であるはずの実装のものであった。**仕様は、書くだけでは検査されない。走らせて、しかも複数の実装で走らせて、初めて検査される。**

再現方法

```
# 本稿の10本を三実装で流す
cd ~/aios/abclcp
for f in docs/samples/paper/s*.aipl; do
  printf 'load %s\ncompile\n' "$PWD/$f" > /tmp/s.repl
  ./src/abclrepl_thread -q -f /tmp/s.repl
done

# 通るプログラムの一致 / 通ってはいけないプログラムの拒否
sh tools/run_all_impls.sh
sh tools/run_reject_tests.sh
```

参考文献

[1] Akinori Yonezawa, Jean-Pierre Briot, Etsuya Shibayama. Object-Oriented Concurrent Programming in ABCL/1. *OOPSLA 1986*, pp. 258–268.

[2] Akinori Yonezawa (ed.). *ABCL: An Object-Oriented Concurrent System*. MIT Press, 1990.

[3] Kenjiro Taura, Satoshi Matsuoka, Akinori Yonezawa. ABCL/f: A Future-Based Polymorphic Typed Concurrent Object-Oriented Language. 1994.

[4] Takuo Watanabe, Akinori Yonezawa. Reflection in an Object-Oriented Concurrent Language (ABCL/R). *OOPSLA 1988*.

[5] Gul Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.

[6] Naoki Kobayashi, Akinori Yonezawa. Type-Theoretic Foundations for Concurrent Object-Oriented Programming. *OOPSLA 1994*.

[7] Naoki Kobayashi, Benjamin C. Pierce, David N. Turner. Linearity and the Pi-Calculus. *POPL 1996*.

[8] Naoki Kobayashi. A Partially Deadlock-Free Typed Process Calculus (1997) / A New Type System for Deadlock-Free Processes (*CONCUR 2006*).

[9] Naoki Kobayashi. A Generic Type System for the Pi-Calculus. *POPL 2001*.

[10] Naoki Kobayashi et al. Resource Usage Analysis. 2001/2002/2005.

[11] Naoki Kobayashi, C.-H. Luke Ong. A Type System Equivalent to the Modal Mu-Calculus Model Checking of Higher-Order Recursion Schemes. 2009.

[12] Naoki Kobayashi et al. Sized Types and Parallel Complexity.

[13] Lightbend. Akka Typed — `Behavior[T]` and behavior replacement.

[14] Ericsson. Erlang/OTP `gen_statem` Behaviour.

[15] 児玉靖司. AIPL: Actor based Intelligent Parallel Language — 設計から実現. 2026 年 8 月 21 日（本稿の第 1 版）.

[16] 児玉靖司. AIPL（ABCLc+）と ABCL/1 — 機能とモデルの観点からの比較. 2026 年 7 月 30 日.
[17] 児玉靖司. AIPL の言語仕様の検討 — ABCL/1 の機構の再導入と、分散 AI エージェント／実時間 OS という目的から. 2026 年 7 月 30 日.
[18] 児玉靖司. AIPL の型健全性と型安全性（第 2 版）. 2026 年 7 月 30 日.
[19] 児玉靖司. AIPL におけるメソッド戻り値型の推論 — reply の引数から型は決まるか. 2026 年 7 月 29 日.
[20] 児玉靖司. AIPL / ABCLc+ ユーザーズガイド. 2026 年 7 月 30 日.
[21] 児玉靖司. 小林研究の成果を AIPL に取り入れる場合の言語仕様 — 可能性、メリット、デメリット. 2026 年 7 月 30 日.